

MONA Protocol Exploit Analysis

Researcher: Ali **Date:** 27 April 2026

How did the MONA Protocol hack happen? - Full Hack Analysis

on 14 april, 2026 the MONA Protocol faaced a technical attack that cost it approximately \$60,000, The attack was not the result of a cryptographic error, but rather a **logical decoupling** in the burn mechanism. The attacker exploited a vulnerability that allowed them to collect the sale proceeds first, and then return later to remove the ‘sold goods’ from the pool, which broke the price balance in their favor

The exploit was executed through three primary phases:

Phase 1: Deferred Burn Accounting (The IOU Flaw)

- The MONA Protocol was designed as a deflationary token model. The core mechanism was intended to burn a percentage of every `sell` transaction to reduce the total supply and increase token scarcity over time.
- A critical design flaw exists within the `_handleSell()` function. Instead of executing an Atomic Inline Burn—where tokens are immediately subtracted and destroyed from the seller’s balance during the swap transaction—the protocol defers this action. It records the burn amount as a “credit” or “debt” within the `BurnAddress.sellMona` state variable.
- his accounting decoupling allowed the attacker to “stuff” the system with massive amounts of pending burn credits. By utilizing the 10,000 MONA tokens previously accumulated through the Sybil accounts, the attacker performed a series of strategic sells. This increased the `sellMona` value to approximately 9,900 MONA in deferred credits. At this stage, the system viewed these tokens as “burned” on the ledger, even though the actual MONA inventory remained physically present within the Liquidity Pool’s reserves.

```

// contracts/mona/monaToken.sol
// Verified source, lines 145-152 and 181-207

function _update(address from, address to, uint256 amount) internal override {
    require(!isBlacklisted[from] && !isBlacklisted[to], "Blacklisted");
    if (isExcludedFromTransfer[from] || isExcludedFromTransfer[to]) {
        super._update(from, to, amount);
        return;
    }

    if (from == lpPairAddress) {
        _handleBuy(from, to, amount);
    } else if (to == lpPairAddress) {
        _handleSell(from, to, amount);
    } else {
        super._update(from, to, amount);
        IburnAddressInterface(burnAddress).burn();
        // VULNERABILITY: any non-pair MONA transfer can settle old sell credits later,
        // at attacker-chosen timing, even if this transfer moves 0 tokens.
    }
}

function _handleSell(address from, address to, uint256 amount) private {
    require(isOpenTrade, "Not open");
    require(block.timestamp - lastTradeTime[from] >= COOLDOWN_TIME, "Cooldown");

    // ... fees omitted ...

    uint256 remainingMona = amount - nodeFee - ecologyFee - remainingBurn;
    (uint256 monaReserve, uint256 usdtReserve, ) = _getPairReserves();
    uint256 amountUSDT = getAmountOut(remainingMona, monaReserve, usdtReserve);
    uint256 monaDeduction = _handleProfitDeduction(from, amountUSDT, monaReserve, usdtReserve);
    uint256 monaTransfer = remainingMona - monaDeduction;

    super._update(from, to, monaTransfer);
    _updateLastTradeTime(from);
    IburnAddressInterface(burnAddress).sell(monaTransfer);
    // VULNERABILITY: the pair pays USDT for this sell now, but `monaTransfer`
    // is also queued for a future burn from the LP itself.
}

```

Phase 2:

- The protocol logic contains a vulnerability within the `_update()` function, which serves as a global hook for all token transfers. This function was programmed to invoke `burnAddress.burn()` whenever a transfer occurred that did not involve the `lpPair` address. The attacker exploited this by executing a `transferFrom(0xdead, 0xdead, 0)`, a transaction that satisfied the conditional branch required to trigger the burn mechanism. Because the transfer value was zero, the Ethereum Virtual Machine did not require any token allowance or balance, allowing the call to succeed without cost. This step functioned as an external trigger, giving the attacker total control over the timing of the burn settlement. By synchronizing this trigger with a specific block height where the liquidity pool was vulnerable, the attacker forced the protocol to initiate the final reserve manipulation that led to the collapse of the token's price balance.

```

// contracts/mona/burnAddress.sol
// Verified source, lines 44-58

function burn() external {
    if (msg.sender == monaTokenAddress) {
        uint256 amount = sellMona - burnedMona;
        if (amount == 0) {
            return;
        }

        uint256 currentBurned = monaToken.balanceOf(address(0xdead));
        uint256 burnLimit = monaToken.burnLimit();
        if (currentBurned < burnLimit) {
            uint256 needBurn = burnLimit - currentBurned;
            if (needBurn < amount) {
                amount = needBurn;
            }
            burnedMona += amount;
            monaToken.burnsellMona(amount);
            // VULNERABILITY: burns from the LP based on stale cumulative sell credits,
            // not as part of the sell that created those credits.
        }
    } else {
        tapcount++;
    }
}

```

```

// contracts/mona/monaToken.sol
// Verified source, lines 284-295

function burnsellMona(uint256 amount) external {
    require(amount > 0, "Must be greater than 0");
    require(msg.sender == burnAddress || msg.sender == joinAddress, "Only burnAddress or joinAddress");

    super._update(lpPairAddress, address(0xdead), amount);
    // VULNERABILITY: directly mutates the pair's MONA balance after an unrelated trade.

    IPancakePair(lpPairAddress).sync();
    // VULNERABILITY: commits the manipulated balance to AMM reserves.

    emit extracTransfer(lpPairAddress, address(0xdead), amount);
}

```

Phase 3: Flash Loan Execution and Reserve Manipulation In the final stage of the attack, the exploiter utilized a flash loan to acquire a significant amount of USDT, which was then used to purchase a massive quantity of MONA tokens directly from the liquidity pool. This strategic buy-back served a dual purpose: it drained the pool's MONA reserves to a critically low level and concentrated the token supply into the attacker's hands. At this precise moment, the attacker triggered the deferred burn mechanism from Phase 2. Because the sellMona amount recorded in Phase 1 was based on earlier, much larger volumes, the protocol

attempted to burn a quantity of tokens that was now disproportionately huge compared to the depleted reserves currently in the pool. The contract manually removed these tokens from the pool and called the `sync()` function, which forced the automated market maker to recognize the near-zero inventory of MONA while the USDT balance remained high. This collapsed the internal price calculation, causing the value of the attacker’s MONA tokens to skyrocket instantly. The attacker then sold their MONA holdings back into the manipulated pool, extracting the entire USDT liquidity and completing the drain of approximately 60,950 USDT.

5. Final Conclusion The MONA Protocol hack serves as a prime example of how asynchronous accounting can lead to total pool depletion. By decoupling the burn mechanism from the swap execution, the protocol introduced a “timing window” that allowed an attacker to manipulate price reserves manually. For future DeFi deployments, all tax and burn actions must be atomic and strictly integrated into the swap logic to maintain the integrity of the constant product invariant.